Curs 3 – Workloads

Lecture objectives

1. Types of workloads
2. Workloads selection

Test workload types

- Real workload
    - System being used for normal operations (by actual people)
- Synthetic workload
    - Similar to real workload
    - Can be applied repeatedly in a controlled manner
    - No large real-world data files
    - No sensitive data
    - Easily modified without affecting operations
    - Easily ported to different systems due to its small size
    - May have built-in measurement capabilities

Desired characteristics of a workbench

- Representative
    - of an application area
- Portable
    - run on many different architectures and implementations
- Unbiased
    - not designed to favour a particular system
- Scalable
    - run on both small and large systems
- Measurable
    - easy to measure => broad acceptance
- Repeatable
    - minimum variance
- Explainable
    - single number easy to understand

Workloads for computer systems

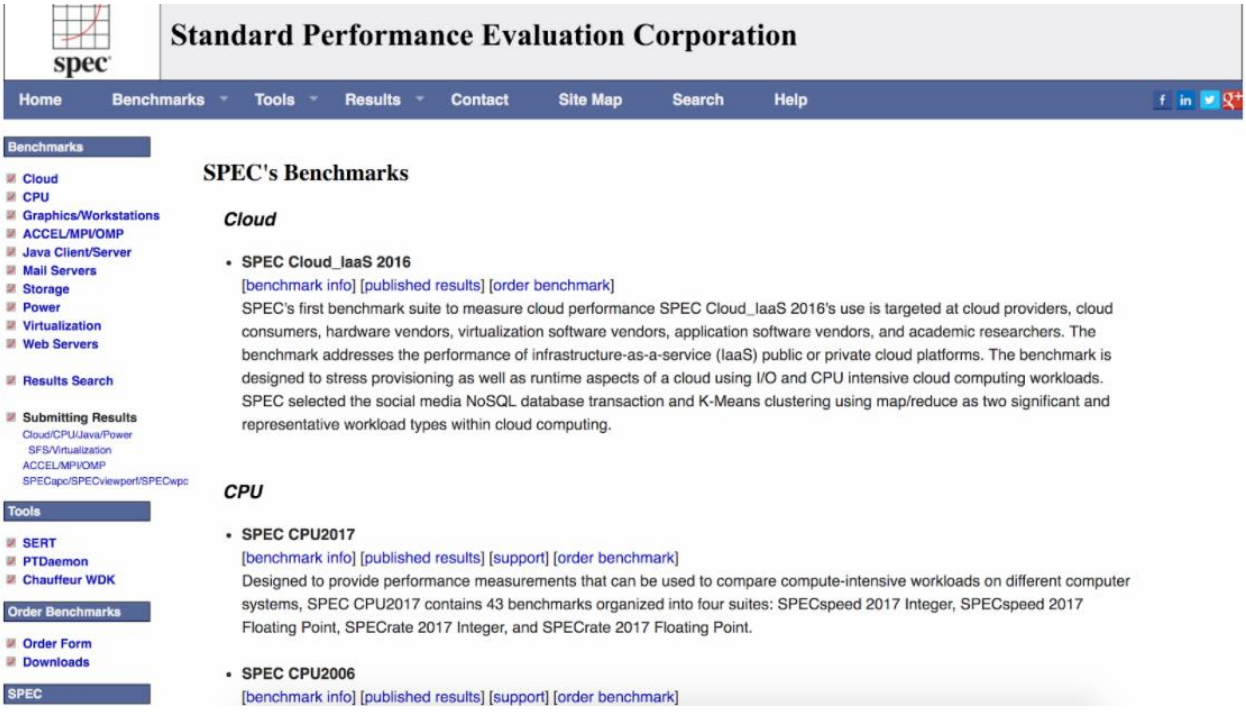
1. Add instructions
2. Instruction mixes - usage frequency
3. Kernels - matches order - micro-benchmark

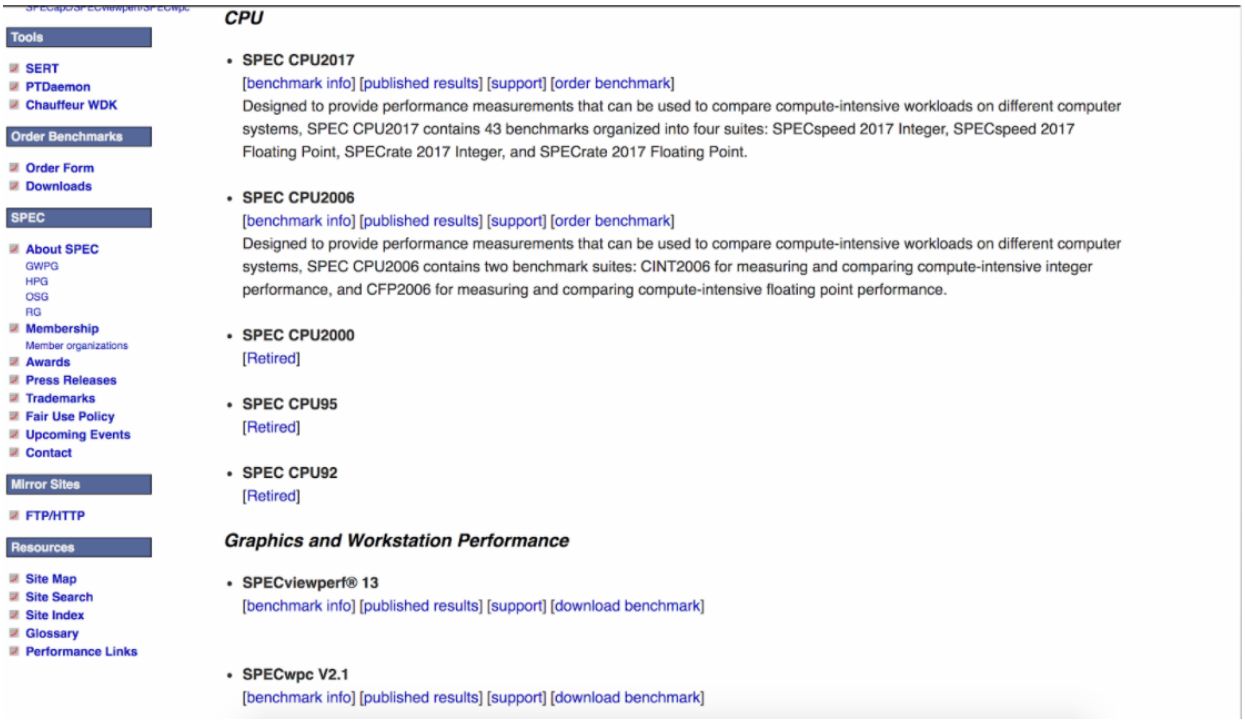
4. Synthetic programs - bigger programs
5. Application benchmarks - matches I/O

Synthetic programs

- Synthetic programs
    - Whetstone (1972) - floating point intensive numerical code
    - Linpack (1976) - linear algebra package - still used - top5000.org to rank supercomputers
    - Dhrystone (1984) - integer only, system code
- Vendor specific benchmarks
    - WinSAT (Windows System Assessment Tool)
        - Control Panel —> Performance Information and Tools
        - Reports Windows Experience Index (WEI)
        - Five sub-scores: processor, memory, 2D graphics, 3D graphics, disk
        - Overall = min of five sub-scores o SAP, Lotus/IBM, Oracle, etc
    - Use binary / machine code
- Industry benchmarks associations
    - SPEC (Systems Performance Evaluation Cooperative)
    - BAPCo (Business Applications Performance Corporation)
    - EEMBC (The Embedded Microprocessor Benchmark Consortium)
    - TPC (Transaction Performance Council) - database systems
    - SPC (Storage Performance Council) - storage systems
- https://en.wikioedia.org/wiki/Benchmark (computing)

SPEC Benchmarks





SPEC CPU benchmark

- Consists of many programs
- Ported to various platforms to remove dependency on I/O or Oss
- Compilation and execution is automated
    - Teste supplies a configuration file with location of C compiler, compilation flags, etc
- Speed
    - Execution time on the system normalised by a well known system
- Geometric mean of ratios is reported as SPECint or SPECfp
- Rate - throughput - jobs/s. Geometric mean = SPECrate

Benchmark challenges / issues

Main problem - optimising/tuning to score higher on benchmarks

- Optimised compilations not representative of real-world
    - Baseline and peak numbers (vendors publicise only peak numbers)
- n-CPU systems run n copies in parallel
    - Not representative since there is no queue of waiting jobs
- Source codes of commonly used programs are not available for use in benchmarks
    - Mostly open source
- Long run times allow jobs to run from cache
    - Not representative

- High performance on high priced systems
    - Price/performance ratio

Summary

- Benchmarks have moved from instructions (MIPS), kernels, synthetic benchmarks to industry standard benchmarks.
- Benchmarks should be representative, portable, unbiased, scalable, measurable, repeatable and explainable.
- Difficult to ensure representativeness of benchmark (compiler flags, run time, configurations - can be manipulated)
- Price and energy consumption are important along with performance

Workload Selection

Workload Selection

- Services exercised
- Level of details
- Loading level
- Timeliness

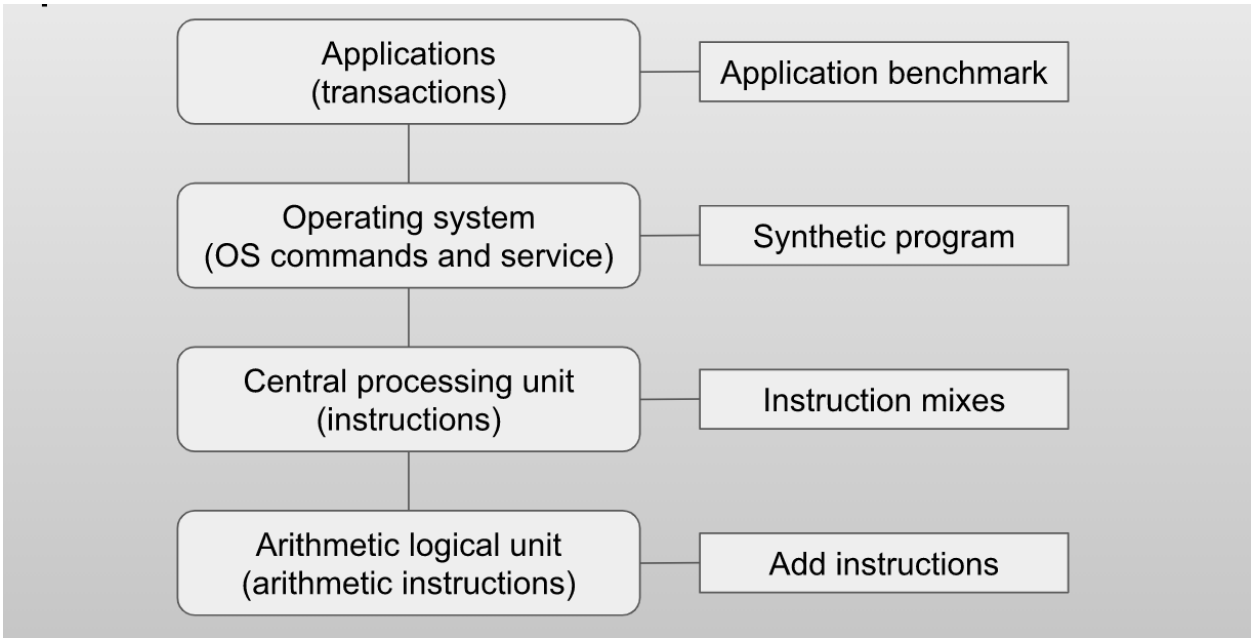
Services exercised

System services determine the workload and metrics

System <- Services <- Metrics and Workload

- System Under Test (SUT) vs Component Under Study (CUT)
- Metrics depend upon SUT
    - MIPS is ok for 2 CPUs, but not for 2 time-sharing systems
- Workload depends upon SUT
- Examples
    - CPU – instructions
    - System - transactions
    - Multiple services - exercise as complete a set of services as possible

Example



Level of detail

- Most frequent operations
  - Only if one service is significantly more frequent than others
  - Example: Add instructions
- Frequency of request types
  - Example: Instruction mixes
- Time-stamped sequence of requests
  - Probably too detailed
  - May require exact reproduction of component behaviour
- Average resource demand
- Distribution of resource demands

Representativeness

- The test workload and the real workload should have the same:
  - Elapsed time
  - Resource demands
  - Resource usage profile - sequence and amounts in which the different resources are used

Timeliness

It has to represent today's usage

Other considerations

- Loading level - a workload may exercise a system to its:
    - Full capacity (best case)
    - Beyond its capacity (worst case)
    - At a load level observed in real world (typical case)
- Impact of external components
    - Do not use a workload that makes external component a bottleneck => all alternatives to this system give equally good performance.
- Repeatability

Summary

- Services determine the workload (and metrics)
- Level of detail of the workload should match that of the model being used
- Workload should be representative of the real systems' usage in recent past